Cognitive Telemetry & Activity Block Engine

Detailed Architecture, Extensible Scientific Dimensions & Implementation Plan

Architecture Blueprint | Extensible Strategy Pattern | Privacy-by-Design | FastAPI & Local-First SQLite

1. Executive Summary & Design Principles

This project implements a lightweight, local-first Python telemetry daemon and REST API designed to help knowledge workers—specifically individuals with ADHD—gain empirical self-insight into how they work. The engine operates on three uncompromised pillars:

- **Empirical Objectivity:** Rejects speculative, subjective inferences (e.g., direct mental load or emotional mood). Only physical, scientifically verifiable behavior is collected and rolled up into structured *Activity Blocks*.
- **Privacy by Construction:** No raw keystroke content or characters are ever captured, buffered, or stored. Window titles and URL parameters are sanitized to executable names or domain roots. The service runs strictly bound to `127.0.0.1`.
- **Pluggable Dimension Architecture:** Telemetry is strictly categorized into 4 baseline scientific dimensions. The system employs an open-closed Provider/Strategy Pattern allowing new dimensions (e.g., posture, ambient sound) to be added without modifying the chunker, storage models, or API endpoints.

2. The 4 Scientific Dimensions (Formulas & Research Grounding)

All data points collected and exposed belong exclusively to one of four empirical dimensions, each tied to validated psychomotor and cognitive literature:

Dimension	Concrete Metrics	Mathematical Formulation	Scientific Literature
1. Temporal Dynamics	Active seconds, idle seconds, continuous block duration, micro-pause rate (1–5s stops).	$Duty = T_{active} / T_{total}$	**Ultradian Rhythms** (Kleitman, 1963) & Executive Resource Depletion (Baumeister et al.). 90-minute sustained attention boundary.
2. Attentional Fragmentation	Context switches per minute, dwell times, application Shannon entropy.	$H = -\Sigma\, p_i\, ln(p_i)$	**Attention Residue Theory** (Leroy, 2009) & Task Interruption Costs (Mark et al., 2005). Impairment from frequent task swaps.
3. Psychomotor Mechanics	Inter-keystroke interval (IKI) coefficient of variation, correction ratio (backspaces/deletes).	$CV = \sigma_{IKI} / \mu_{IKI}$ $R_{corr} = K_{del} / K_{total}$	**Typing Biomechanics & Cognitive Stress** (Yamada, 1980; Vizer et al., 2009; Sun et al.). Motor slip increases with mental fatigue.
4. Circadian & Cumulative Load	Diurnal hour slotting, post-prandial dip flag, cumulative daily screen time without >30m reset.	$T_{cum} = \Sigma\, Duration_i$	**Circadian Chronobiology** (Roenneberg et al.). 13:00–15:00 physiological alertness dips and chronic time-on-task decay.

3. System Architecture & The Extensibility Model

The architecture splits responsibilities cleanly between signal ingestion, metric computation, activity chunking, and HTTP delivery.

> **How Extensibility Works: The BaseDimension Contract**
>
> Every dimension is an independent Python class inheriting from `BaseDimension`. To introduce a new dimension in the future (e.g., `BiomechanicalDimension` tracking webcam posture or `AmbientDimension` tracking decibels), a developer simply writes a new subclass and registers it in `DimensionRegistry.register(new_dim)`. The API, database schemas, and chunker loop remain untouched.

```
# app/dimensions/base.py
from abc import ABC, abstractmethod
from typing import Dict, Any, List
from pydantic import BaseModel

class InsightPrompt(BaseModel):
    dimension_name: str
    observation: str           # Objective, factual statement of behavior
    scientific_mechanism: str  # Validated research citation / theoretical basis
    suggested_action: str      # Low-friction, non-judgmental adjustment

class BaseDimension(ABC):
    @property
    @abstractmethod
    def name(self) -> str:
        """Unique key identifying this dimension (e.g. 'temporal')."""
        pass

    @abstractmethod
    def reset_epoch(self) -> None:
        """Clears temporary rolling counters for the next 30-second epoch."""
        pass

    @abstractmethod
    def on_event(self, event_type: str, data: Dict[str, Any]) -> None:
        """Processes real-time hardware/OS ticks dispatched by collectors."""
        pass

    @abstractmethod
    def compute_epoch_metrics(self) -> Dict[str, Any]:
        """Calculates 30-second micro-summary for this dimension."""
        pass

    @abstractmethod
    def aggregate_block_metrics(self, epoch_histories: List[Dict[str, Any]]) -> Dict[str, Any]:
        """Fuses all 30-second epoch dictionaries into the finalized block payload."""
        pass

    @abstractmethod
    def evaluate_scientific_insights(self, block_metrics: Dict[str, Any], duration_sec: float) -> List[Insi
        """Tests empirical values against research thresholds to generate suggestions."""
        pass
```

4. Activity Block Chunking & Debounce Logic

A primary challenge in desktop telemetry is preventing noisy micro-interruptions (such as skipping a song in Spotify for 4 seconds) from fracturing clean work sessions into dozens of meaningless fragments. The **Chunking Engine** enforces a stateful hysteresis rule:

- **Epoch Collection:** Hardware listeners feed raw event counts into the active dimensions every 30 seconds.

- **Transient Interruption Buffer:** If the user switches away from their primary application, the engine marks a provisional state change. If the user returns within 60 seconds, the brief excursion is absorbed into the existing block and logged merely as an internal context switch.
- **Definitive Block Boundary:** If a different application or an idle state persists for ≥90 seconds, the current block is sealed:
  1. `end_time` is pinned to the exact transition timestamp.
  2. Each dimension in `DimensionRegistry` aggregates its history for that span.
  3. Dimension-specific `evaluate_scientific_insights()` routines run.
  4. The completed `ActivityBlock` is committed to the local SQLite database.
  5. A new empty block begins accumulating the incoming activity.

5. Polymorphic Database & Pydantic Schema

Because dimensions are polymorphic dictionaries, the database schema never requires column migrations when a dimension is added or updated.

```
-- SQLite Schema Definition
CREATE TABLE activity_blocks (
    id TEXT PRIMARY KEY,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    duration_seconds REAL NOT NULL,
    primary_process TEXT NOT NULL,
    dimensions_json TEXT NOT NULL,         -- JSON map of all registered dimensions
    triggered_insights_json TEXT NOT NULL, -- JSON list of generated insight prompts
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_blocks_time ON activity_blocks(start_time, end_time);
```

Sample Finalized Block JSON Output

```
{
  "id": "blk_20261027_094500",
  "start_time": "2026-10-27T09:45:00Z",
  "end_time": "2026-10-27T10:47:00Z",
  "duration_seconds": 3720.0,
  "primary_process": "Code.exe",
  "dimensions": {
    "temporal": { "total_active_seconds": 3120.0, "duty_cycle_ratio": 0.838, "micro_pause_rate_per_min": 1.4 },
    "attentional": { "total_switches": 8, "switches_per_minute": 0.13, "process_entropy": 0.312 },
    "psychomotor": { "total_keystrokes": 1840, "correction_ratio": 0.082, "mean_rhythm_stability": 0.28 },
    "circadian": { "end_hour": 10, "is_circadian_dip_window": false, "cumulative_work_hours_today": 1.03 }
  },
  "triggered_insights": []
}
```

6. REST API Endpoints Specification

The service exposes a streamlined FastAPI interface running locally on port `8765`:

Endpoint	Method	Query / Params	Description
`/api/v1/status`	`GET`	None	Returns real-time status: currently active application, live block elapsed time, and real-time switch rate over the past 5 minutes.

Endpoint	Method	Query / Params	Description
`/api/v1/blocks`	`GET`	`start_date`, `end_date`, `limit`	Returns historical finalized activity blocks for timeline visualizations and session review.
`/api/v1/blocks/{id}`	`GET`	`id`	Returns granular metric breakdowns and dimension JSON for a specific block.
`/api/v1/summary/today`	`GET`	None	Aggregates all 4 dimensions across the current day (total screen time, average entropy, average correction ratio).
`/api/v1/insights/active`	`GET`	None	Surfaces any currently active science-backed intervention suggestions based on the ongoing session state.

7. Complete Project Directory Layout

```
telemetry-engine/
├── app/
│   ├── __init__.py
│   ├── main.py                  # FastAPI entry point & daemon lifespans
│   ├── config.py                # Epoch interval (30s), debounce timeout (60-90s)
│   ├── collector/               # Hardware listeners (Zero-text privacy)
│   │   ├── keyboard_listener.py   # Keystroke counts, interval deltas, correction keys
│   │   ├── mouse_listener.py      # Click counts, distance moved
│   │   └── window_resolver.py     # Active process name only (sanitized)
│   ├── dimensions/              # Modular Scientific Dimensions
│   │   ├── base.py              # BaseDimension interface & InsightPrompt schema
│   │   ├── registry.py          # DimensionRegistry (orchestrates and broadcasts)
│   │   ├── temporal.py          # Dim 1: Duty cycle, duration, micro-pauses
│   │   ├── attentional.py       # Dim 2: Shannon entropy, switch rate
│   │   ├── psychomotor.py       # Dim 3: IKI variance, correction ratio
│   │   └── circadian.py         # Dim 4: Diurnal alignment, cumulative screen time
│   ├── engine/
│   │   ├── chunker.py           # Debounce buffer & state-machine
│   │   └── scheduler.py         # 30-second epoch loop runner
│   ├── storage/
│   │   ├── database.py          # SQLite async engine (aiosqlite / SQLAlchemy)
│   │   └── repository.py        # CRUD operations for ActivityBlocks
│   └── api/
│       ├── routes_realtime.py   # Live status and current session
│       └── routes_blocks.py     # History queries & insight endpoints
├── tests/
│   ├── test_dimensions.py       # Unit test for math (Entropy, IKI variance)
│   ├── test_chunker.py          # Test debounce transitions with mock events
│   └── test_api.py              # Verify FastAPI endpoints and JSON formats
├── pyproject.toml
└── README.md
```

8. Phased Implementation Roadmap

1. **Sprint 1: Dimension Interfaces & Math Validation (Days 1–3)**
   Implement `BaseDimension`, `DimensionRegistry`, and the 4 dimension classes. Write unit tests with synthetic event feeds to verify Shannon entropy, IKI variance, and duty cycle calculations.
2. **Sprint 2: Zero-Text Telemetry Collector (Days 4–7)**
   Implement `pynput` keyboard/mouse listeners with strict counter-only logic and process resolution (Windows `win32gui`, macOS `NSWorkspace`, Linux `Xlib`). Ensure character codes are never stored in memory.

3. **Sprint 3: Chunking Engine & SQLite Storage (Days 8–11)**
   Build the 30-second periodic scheduler and the debounce state machine. Verify that transient 10-second app switches do not split blocks, and that genuine >90s shifts correctly seal and persist blocks.
4. **Sprint 4: FastAPI Service & Insight Rules (Days 12–14)**
   Implement the REST endpoints, wire the background daemon into the FastAPI lifespan context, and test API retrieval from a local browser/client.